PAPER NAME

**Advanced_Software_Engineering_Capstone_ALIAH BULA.pdf**

AUTHOR

**ALIAH BULA**

WORD COUNT

**9217 Words**

CHARACTER COUNT

**58148 Characters**

PAGE COUNT

**38 Pages**

FILE SIZE

**716.4KB**

SUBMISSION DATE

**Feb 27, 2026 12:35 AM GMT+8**

REPORT DATE

**Feb 27, 2026 12:37 AM GMT+8**

## 3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

- 3% Internet database
- 1% Publications database
- Crossref database
- Crossref Posted Content database

## Excluded from Similarity Report

- Submitted Works database
- Bibliographic material

# MIT 265 - Advanced Software Engineering

# Capstone Project

Development and Evaluation of an AI-Powered Student Academic Information System with Early Academic Risk Prediction and Intelligent Assistance at Mindanao State University–General Santos

## Master in Information Technology

Second Semester SY 2025-2026

February 28, 2026

Submitted to:

Mrs. Brenda M. Balala, MIT

## Contents

**CHAPTER 1 – INTRODUCTION** ..... 4
1.1 Background of the Study ..... 4
1.2 Problem Statement ..... 4
1.3 Objectives of the Study ..... 4
1.4 Scope and Delimitations ..... 5
1.5 Significance of the Study ..... 5
1.6 Definition of Terms ..... 6
**CHAPTER 2 – REVIEW OF RELATED LITERATURE AND STUDIES** ..... 7
2.1 Theoretical Framework ..... 7
2.2 Conceptual Framework ..... 8
2.3 Related Literature ..... 9
2.4 Related Systems ..... 11
2.5 Research Gap Analysis ..... 12
**CHAPTER 3 – SYSTEM DESIGN AND DEVELOPMENT** ..... 14
3.1 Overview of the System ..... 15
3.2 Software Development Model ..... 16
3.3 System Architecture ..... 17
3.4 System Design Specifications ..... 22
3.5 Data Modeling (ERD + Data Dictionary) ..... 23
3.6 Graph Database Modeling ..... 25
3.7 Algorithms and Analytics ..... 26
CHAPTER 4 – SUMMARY, FINDINGS, AND CONCLUSION ..... 29
4.1 Summary of Findings from the Analysis ..... 29
4.2 Conclusion Based on Models and Frameworks ..... 29
4.3 Implications for Software Engineering Practice ..... 29.
CHAPTER 5 – FUTURE WORK / INNOVATIONS ..... 31
5.1 Potential Enhancements ..... 31
5.2 Long-Term Expansion Opportunities ..... 31
REFERENCES ..... 32
APPENDICES ..... 33
Appendix A – Sample Code Snippets (Optional) ..... 33
Appendix B – Data Dictionary ..... 35

**REFERENCES** ..... Error! Bookmark not defined.

**APPENDICES** ..... Error! Bookmark not defined.

A. Sample Code Snippets ..... **Error! Bookmark not defined.**

B. Data Dictionary ..... **Error! Bookmark not defined.**

# CHAPTER 1 – INTRODUCTION

## 1.1 Background of the Study

Educational institutions have increasingly become difficulties in tracking student performance and interventions to be made to at-risk students in time. In Mindanao State University - General Santos, the present Student Academic Information System (SAIS) is mainly used to enter grades, attendance, and enrolment. Nevertheless, it does not have early academic risk prediction, automated reporting, and intelligent student-faculty support facilities. With the introduction of AI-powered systems, rule-based predictive analytics can be integrated, which measures the performance of the students and issues early warnings to the students who are predisposed to academic failure. Moreover, AI chatbots will be able to answer queries of students and faculty, minimize dynamic activities, and enhance system responsiveness. Moreover, the current software engineering solutions, including the graph model and cloud implementation, allow effectively managing the complex academic relations, including the prerequisites, enrolment patterns, and performance trends, and assure real-time analytics and scalable system design. Advanced analytics, query efficiency, and intelligent data modeling are all capabilities that have been offered by platforms such as ONgDB and knowledge graph technologies and are becoming increasingly applicable in the modern educational software systems. The purpose of this research is to create and test an AI-based Student Academic Information System, which will integrate early risk identification, automated reporting, and intelligent support, which would benefit academic monitoring, decision-making, and general student support at Mindanao State University-General Santos.

## 1.2 Problem Statement

Although the current SAIS has the capabilities, there are several limitations:

1. Failure to identify at-risk students in time, thus interventions are delayed.
2. Time-consuming and error-prone, manual reporting and performance analysis.
3. There was no clever query assistance, and so students and faculty asked redundant questions.
4. Small scalability and performance, which impedes managing complicated relations between students and courses. These areas of weakness also point to quantifiable issues, such as the response time of the system, its efficacy in reporting, and its capabilities to predict risks accurately, which can be mitigated using predictive analytics powered by AI and intelligent help modules.

## 1.3 Objectives of the Study

To create and test an AI-powered Student Academic Information System (SAIS) that incorporates rule-based predictive analytics to identify early academic risk and an AI-based chatbot to assist students and faculty at Mindanao State University-General Santos.

This research will attempt to,

1. Incorporate predictive analytics into the SAIS to help identify at-risk students in terms of low academic achievement by designing and implementing the following system modules:

   a. Authentication Module and User Management.

b. Student Academic Records Management Module.
c. Academic Performance Data Processing Module.
d. Predictive Analytics and Risk Assessment Module Rule-Based.
e. Risk Visualization and Alert Module, Academic.
f. Academic Reporting and Analytics Module is automated.

2. Create an AI chatbot that will help students and faculty respond to both system and academic questions by implementing:

1. Smart Chatbot Module based on AI.
2. Knowledge Management Base and Intent Module.
3. System Assessment and Analytics Module.
4. I: Audit Logging and Data Privacy Module.

3. Automate academic performance reporting and create actionable insights to support academic advising and student, advisor, and administrative decision-making.

4. Assess accuracy, efficiency, and usability of the AI-enhanced modules of the system: predictive analytics and AI chatbot.

**1.4 Scope and Delimitations**

Scope:

- Targeted at the undergraduate students at Mindanao State University-General Santos.
- Adopts rule-based predictive analytics based on GPA, failed subjects, and attendance.
- Offers a chatbot on AI to query students and faculty.
- Produces automated risk reports, alerts, and dashboards.

Delimitations:

- Training of machine learning models is not involved; everything is rule-based.
- UI/UX improvements and physical implementation concerns are not core areas of concern.
- Analysis focuses on accuracy, efficiency, and usability as opposed to long-term adoption.

**1.5 Significance of the Study**

- Students: An early warning, individual advice, and academic achievements.
- Faculty/Advisers: Rapidly discover at-risk students and intervene. Administrators: Make evidence-based choices to enhance retention and planning.
- University: Shows contemporary software engineering techniques, such as AI implementation, graph modeling, and cloud implementation.
- Academia: Provides intelligence to AI-powered student support systems and intelligent academic analytics.

### 1.6 Definition of Terms

- AI-Powered System: Software using intelligent algorithms, including rule-based analytics and chatbots.
- Rule-Based Predictive Analytics: Evaluates data using predefined rules to identify academic risk.
- SAIS: Student Academic Information System
- Chatbot: AI interface for handling student and faculty queries.
- ONgDB: Open-source graph database platform for complex relationship modeling.
- Node/Edge: Graph elements representing entities and their relationships.

# CHAPTER 2 – REVIEW OF RELATED LITERATURE AND STUDIES

## 2.1 Theoretical Framework

Developing an AI-powered Student Academic Information System that can significantly contribute to early risk prediction and provide the necessary intelligent assistance, drawing on various well-established disciplines in software engineering and expert systems. All these spheres influenced the most important choices in the system design, the choice of tools, and the arrangement of its main modules, namely predictive analytics, the AI chatbot, and automated reporting, to work reliably and predictably.

| Theoretical Foundation | Core Principle | Application in This Study |
|---|---|---|
| **Agile Methodology (Schwaber & Beedle, 2002)** | **Iterative development, continuous feedback, and incremental delivery of functional software.** | **Guides the four-sprint development cycle, enabling module-by-module delivery and stakeholder feedback integration.** |
| **DevOps Practices (Bass, Weber & Zhu, 2015)** | **Continuous integration, automated deployment, and system reliability.** | **CI/CD pipeline ensures seamless updates to analytics rules, chatbot knowledge base, and reporting modules.** |
| **Graph Theory (Diestel, 2010)** | **Mathematical modeling of complex relationships between interconnected entities.** | **Informed the relational schema design in MySQL, particularly the foreign key relationships between students, courses, enrollments, and performance records.** |
| **Rule-Based Expert Systems (Buchanan & Shortliffe, 1984)** | **Predefined logical rules for transparent, interpretable inference.** | **Powers the risk analytics engine, classifying students as Low, Moderate, or High risk based on GPA, attendance, and failed subjects.** |

*Table 2.1. Theoretical Foundations and Their Application in the AI-Powered SAIS.*

Though the overall development process was on a waterfall structure, Agile thinking was applied to the way the stakeholder requirements were obtained, as well as the evaluation of the system. Design decisions at each stage were informed by feedback from students, faculty, and administrators and ensured that the system was informed by real user needs, as opposed to assumptions.

Principles of the practical deployment influenced the installation of the system in terms of hosting and maintenance. The local server environment based on XAMPP made it possible to build the system, test it, and run it under a controlled environment, which meant that the modifications of risk rules, the chatbot

knowledge base, or reporting logic could be implemented without interfering with the ongoing user sessions.

The graph theory was an effective conceptualization of how academic information was organized in the research. Instead of considering student records, courses, and enrollments as individual rows in different tables, graph theory makes one think of these objects and relationships, and this directly informed the design of the database relationships and foreign key relationships in MySQL using phpMyAdmin.

The rule-based expert system theory was the most feasible option as far as predicting the risk side of the system is concerned. The machine learning methodologies were also discussed, which raises interpretability issues that would not be easily defensible in an academic advising setting. Through clearly defined regulations on GPA, attendance, and failed classes, advisers and students can know precisely why a red flag was raised, which is immensely important when the consequence is to the academic position of a person.

### 2.2 Conceptual Framework

The general design of the AI-based SAIS is based on the Input-Process-Output (IPO) model. It was selected as it provides a clear view of the flow of data over the system at what is collected, what transpires out of the analysis, what users see, and what they do. It also facilitated easier tracing of each module to a particular section of the workflow in the process of design and testing.

| INPUT | PROCESS | OUTPUT |
|---|---|---|
| **Student Academic Records**<br>○ **GPA and grade records**<br>○ **Failed subject history**<br>○ **Attendance data**<br>○ **Enrollment history**<br><br>**User Inputs & Rules**<br>○ **Predefined academic risk rules**<br>○ **Student & faculty queries**<br>○ **Administrator configurations** | **Analytics & Classification**<br>○ **Rule-based risk level classification**<br>○ **GPA threshold evaluation**<br>○ **Attendance percentage analysis**<br>○ **Failed subject detection**<br><br>**AI Assistance & Reporting**<br>○ **AI chatbot intent matching**<br>○ **Automated report generation**<br>○ **Dashboard computation**<br>○ **Audit log recording** | **Risk Assessment Results**<br>○ **Low / Moderate / High risk classification**<br>○ **Early warning alerts**<br>○ **Adviser notifications**<br><br>**Intelligent Support & Analytics**<br>○ **Chatbot responses & guidance**<br>○ **Interactive analytics dashboards**<br>○ **Automated academic reports**<br>○ **Exportable performance data** |

*Figure 2.1. IPO Conceptual Framework of the AI-Powered SAIS for Early Academic Risk Prediction and Intelligent Assistance at MSU–General Santos.*

The conclusion of it all that can be seen by the framework is that the system does not simply store academic records- it does something with them to make them useful. Risk classifications, alerts, chatbot replies, and reports are the in and out data, grades, attendance figures, and enrollment data. This entire flow was kept in mind during development to ensure that there was no single functional requirement that was not linked to an actual output.

### 2.2.1 Mapping Theoretical Concepts to System Design

It is reflected in the table below; each theoretical foundation is directly related to a piece of the system that was constructed in reality:

| Theory | IPO Stage | System Module |
|---|---|---|
| **Waterfall SDLC** | **Process (Iterative Development)** | **All modules – 4-phase Waterfall development cycle** |
| **DevOps / CI/CD** | **Process (Deployment Pipeline)** | **GitHub Actions CI/CD pipeline** |
| **Graph Theory** | **Process (Data Modeling)** | **MySQL relational database layer** |
| **Rule-Based Expert Systems** | **Process (Analytics)** | **Rule-Based Risk Analytics Engine** |
| **Expert Systems (Output)** | **Output (Alerts)** | **Alert & Notification Module** |

*Table 2.2. Mapping of Theoretical Frameworks to System Modules.*

## 2.3 Related Literature

The examination of current studies in these disciplines assisted in making clear that the gaps exist and what has already been achieved. The literature in software engineering techniques, education analytics, database design, and intelligent systems each brought something different to the frame of this study, and where it attempts to take it further.

### 2.3.1 Structured SDLC and Information System Development

Research-based on the systematic SDLC methods indicates that modularity and feedback loops enhance the quality and longevity of the software (Pressman, 2014). Agile or Scrum implementation has resulted in well-tested, manageable codebases. The issue, however, is that most student information systems based on such approaches end up being record-keeping systems, they store grades and attendance, and then simply leave said data there. No layer can automatically look at whether a student is not keeping pace, or mechanisms to enable students to find instant answers without having to email their advisor. The current study fills this very gap directly, as a risk engine and a chatbot are integrated at the very beginning and are not considered as either supplements or extras to the system.

### 2.3.2 Entity-Relationship Modeling and Graph Database Technologies

Relational databases have been the default when it comes to student records, and it is understandable; they are efficient with structured and transactional data (Diestel, 2010; Angles and Gutierrez, 2008). All academic information in this study, such as student records, grades, enrollments, attendance, and risk assessments are based on MySQL, which should be administered using phpMyAdmin. The academic relationship queries required to be able to classify risks and to monitor performance do not necessitate a

separate graph database and can be well supported using a properly indexed relational schema containing foreign keys.

### 2.3.3 Predictive Analytics and Early Risk Detection in Education

Machine learning and rule-based systems have been applied to identify at-risk students, and educational data mining literature can record a variety of methods (Romero & Ventura, 2010). ML models tend to work well when provided with sufficient labeled data, but there is an actual issue with them as far as institutional use is concerned: the adviser and the student do not have easy access to how and why a model has flagged a student as high-risk. Such a deficiency of transparency is a grave problem in academic contexts where interventions carry repercussions. Instead, rule-based expert system reasoning is explicit (Buchanan and shortliffe, 1984) - when the GPA is below 2.0, a flag is raised, and that justification can be presented directly to the student. In this research, there could not be any compromise on interpretability.

### 2.3.4 Centralized Data Architecture, Dashboards, and Visualization

In general, dashboard research in pedagogical contexts indicates that the availability of data in a single place, structured, visualized, and role-specifically allows making decisions (Chen et al., 2012). These implementations are limited by the fact that most of them are based on the periodically generated reports that are manually activated. A report must be run and reviewed by the administrator or adviser who is interested in knowing which students are already at risk. The system that was created to obtain this study replaces this model with live dashboards, which update when records are altered, in addition to automatic notices and a chatbot, which can answer questions without someone manually sorting through the data.

### 2.3.5 Cloud Deployment, Scalability, and Hybrid Analytics

The research on cloud and hybrid deployments highlights the actual benefits of scalability and cross-platform access (Stonebraker et al., 2018), but also indicates a common issue: it is not easy to add intelligent components such as analytics engines or chatbots to existing systems without adversely affecting the performance. In the case of this research, the XAMPP local server environment was employed to make the server setup simple and easy to manage, and all the data operations were performed through MySQL using phpMyAdmin. This would make the system sustainable and fit the institutional context at MSU -General Santos.

| Research Area | Key Finding | Gap/Limitation | This Study's Response |
|---|---|---|---|
| **SDLC & IS Development** | **Agile SDLC improves software quality** | **Lacks real-time AI analytics** | **Embeds analytics and chatbot into Agile sprints** |
| **ER & Graph Databases** | **Graph DBs excel at relationship modeling** | **Limited real-time predictive analytics** | **MySQL + phpMyAdmin with rule-based risk engine** |
| **Predictive Analytics** | **ML achieves high accuracy** | **Poor interpretability; needs large data** | **Rule-based system ensures transparency** |

| | | | |
|---|---|---|---|
| **Dashboards & Visualization** | **Dashboards support decision-making** | **Static reports, no AI interaction** | **Dynamic dashboards with chatbot integration** |
| **Cloud & Scalability** | **Cloud improves accessibility** | **Difficulty integrating intelligent modules** | **XAMPP + MySQL + phpMyAdmin architecture** |

*Table 2.3. Synthesis of Related Literature – Findings, Gaps, and Study Responses.*

### 2.4 Related Systems

When examining the way similar systems have been constructed, there is a relatively similar trend: most are sound in terms of storing and displaying records of the academic process, but weak in terms of analysis, automation, or smart interaction. Below, our discussion concentrates on what various types of systems do architecturally correctly, and where they fail at this every time, without mentioning particular commercial products.

#### 2.4.1 Conventional Student Information Systems

The most prevalent and typical form of student information system is an assembly of modules, including enrollment, grading, and reporting modules, all running on a relational database and presented via a web portal. These systems are suitable for their intended purpose: to ensure the organization of records and make them available by role. However, they were not created with the premise that it is the system itself that should be monitoring issues. The engine is not checking whether the grades and attendance of a student are going in the wrong direction, and there is no option for a student to ask a question to the system and get an answer that is prompt and handy.

#### 2.4.2 Systems with Analytics and Dashboard Integration

Other systems have gone beyond simple displays of records and have included analytics dashboards. They are helpful; it helps to be able to look at grade distribution or attendance patterns briefly and make a difference to the advisers. The weakness is that such a dashboard is generally fed using batch processes that run after a definite period, implying that the data depicted may be hours or days old. Also, nearly none of these category systems integrate the use of dashboards and automated alerting/AI assistance. This paper brings these three to a single platform: live dashboards, automatic risk alerts, and a chatbot.

#### 2.4.3 Graph-Based and Hybrid Systems

Other previous systems have investigated the graph database methods in modeling academic relationships. Nonetheless, this paper is pragmatic in that it wholly depends on MySQL, which is run using phpMyAdmin, and well-organized relational tables with well-established foreign key relations that reflect student-course, student-adviser, and student-enrollment relationships. This will ensure that the system is simple, maintainable, and that it fits well within the institutional setting within MSU- General Santos.

| System Type | Architecture Strength | Design Limitation | Addressed By This Study |
|---|---|---|---|
| **Conventional SIS** | **Stable relational data management** | **No analytics, no AI support** | **AI chatbot + rule-based risk engine** |
| **Dashboard-Integrated SIS** | **Visual decision support** | **Static batch reports only** | **Real-time dashboards + automated alerts** |
| **Graph-Based SIS** | **Efficient relationship querying** | **Limited real-time integration** | **MySQL + phpMyAdmin + analytics pipeline** |

*Table 2.4. Comparative Analysis of Related System Architectures.*

### 2.5 Research Gap Analysis

Collected collectively, what the literature and available systems demonstrate is that no single solution has been able to integrate early risk detection, intelligent query support, relationship modeling using graphs, and a stable deployment pipeline in a single coherent academic information system. All of them are located somewhere; however, they have never been united in such an institution as MSU-General Santos. The following are the gaps that this study was specifically designed to solve.

#### 2.5.1 Identified Gaps in Existing Systems

| # | Gap Identified | Impact on Academic Monitoring | This Study's Innovation |
|---|---|---|---|
| **1** | **Absence of early academic risk prediction** | **Students at risk are identified only after failure, delaying intervention** | **Rule-based risk engine classifying students per semester into Low, Moderate, and High risk** |
| **2** | **No intelligent query assistance** | **Faculty and students waste time on repetitive manual inquiries** | **AI chatbot with intent-matched knowledge base providing instant academic guidance** |
| **3** | **Lack of scalable cloud graph deployments** | **Large academic datasets cannot be queried efficiently for complex relationships** | **Well-structured MySQL relational schema with indexed foreign keys enabling efficient academic relationship queries** |

| | | | |
|---|---|---|---|
| **4** | **Limited analytics pipeline integration** | **Performance data remains siloed, preventing unified risk analysis and reporting** | **Unified analytics pipeline connecting MySQL records to the risk engine, reporting module, and dashboards** |
| **5** | **Absence of CI/CD for academic systems** | **System updates are manually deployed, risking downtime and inconsistency** | **GitHub Actions CI/CD pipeline with automated testing and zero-downtime rolling deployment** |
| **6** | **Static, non-interactive reporting** | **Administrators rely on manually generated PDF reports with no drill-down capability** | **Interactive real-time dashboards with exportable analytics for students, advisers, and admins** |

*Table 2.5. Research Gap Analysis – Identified Gaps and Study Innovations.*

All these lapses have their price; some students ought to be red-flagged but are not, advisers waste time taking the same test repeatedly, and administrators work on old-fashioned reports. These issues informed the creation of the system created in this paper by relying on MySQL through phpMyAdmin to perform dependable data handling, a rule-based risk engine, and an AI chatbot to close the monitoring and assistance gaps observed at MSU-General Santos directly.

## CHAPTER 3 – SYSTEM DESIGN AND DEVELOPMENT

### 3.1 Overview of the System

The developed AI-powered SAIS at MSU General Santos is a web-based service constructed based on a simple yet significant notion that a student information system does not serve as the place to keep records, but rather enables people to act on them. The system contains a MySQL relational database to be managed using phpMyAdmin in daily academic data management, a risk engine that operates automatically every semester, and a chatbot that students and faculty can use to obtain quick answers without waiting until office hours.

#### 3.1.1 System Users and Roles

| User Role | Primary Functions | Key Modules Accessed |
|---|---|---|
| **Student** | View grades, attendance, enrollment status; check risk level; use chatbot; receive alerts | Academic Records, Risk Dashboard, Chatbot, Alerts |
| **Faculty Adviser** | Monitor at-risk students; review performance reports; send interventions | At-Risk List, Student Performance View, Reporting, Alerts |
| **Administrator** | Manage users, courses, programs; run risk assessments; manage knowledge base; view audit logs | User Management, Course Management, Risk Engine, KB Manager, Audit Logs |

*Table 3.1. System Users, Functions, and Module Access.*

#### 3.1.2 Core System Functionalities

- User Management and Authentication: Role-based login and access control for students, advisers, and administrators.
- Student Academic Records Management: CRUD operations for enrollment, grade, and attendance data.
- Rule-Based Risk Analytics Engine: Automated classification of academic risk using predefined GPA, attendance, and failure thresholds.
- Alert and Notification Module: Automated dispatch of risk alerts to at-risk students and their assigned advisers.
- AI-Based Intelligent Chatbot: Intent-matched responses to student and faculty academic queries using a curated knowledge base.
- Automated Reporting and Analytics Dashboards: Real-time and exportable performance summaries for all user roles.
- Audit Logging and Data Privacy Module: Comprehensive logging of all system events for compliance and accountability.

## 3.2 Software Development Model

This system was developed under the Waterfall SDLC development model. This choice came about due to the type of project it was. The requirements were clear initially, the institutional setting was stable, and no form of mid-development pivoting that iterative models are formulated was necessary. The sequential approach of working requirements, design, implementation, testing, and deployment also helped make the process of documenting each step straightforward, which is important when working on a thesis where the process must be transparent and reproducible.

**Design Methodology**

Presentation Layer (Front-End)

What the user sees and interacts with:

- Student, Adviser, and Admin dashboards
- Chatbot interface
- Grade and attendance viewers
- Alert notification center

Application Layer (Back-End)

The brain of the system — all business logic runs here:

- Risk Analytics Engine
- AI Chatbot Processor
- Reporting Engine
- Authentication & Access Control (JWT + RBAC)
- Alert Dispatcher
- Audit Logger

Data Layer

Where all data is stored and retrieved:

- MySQL (via phpMyAdmin) — for structured records like grades, enrollments, attendance
- ONgDB — for graph relationships like prerequisites and adviser-student networks

**Development Model**

It was created in an incremental way whereby development of each of the major modules of this system, including the user authentication, academic records management, rule-based risk analytics engine, AI chatbot, alert notification, automated reporting, and audit logging were developed, tested, and validated separately before becoming a part of the main system. The database has been constructed and maintained with phpMyAdmin, which ensured that all the relational structures and the relationships of the data were well laid out in each phase of development. The stability and backward compatibility between the modules were ensured by constant testing done at every point of integration, where the new parts of the system would not interfere with the existing parts of the system. The method enabled the AI-based Student Academic Information System to expand continuously and consistently, with every step upon a tested and stable base, until the entire system was completely assembled and could be installed in Mindanao State University -General Santos.

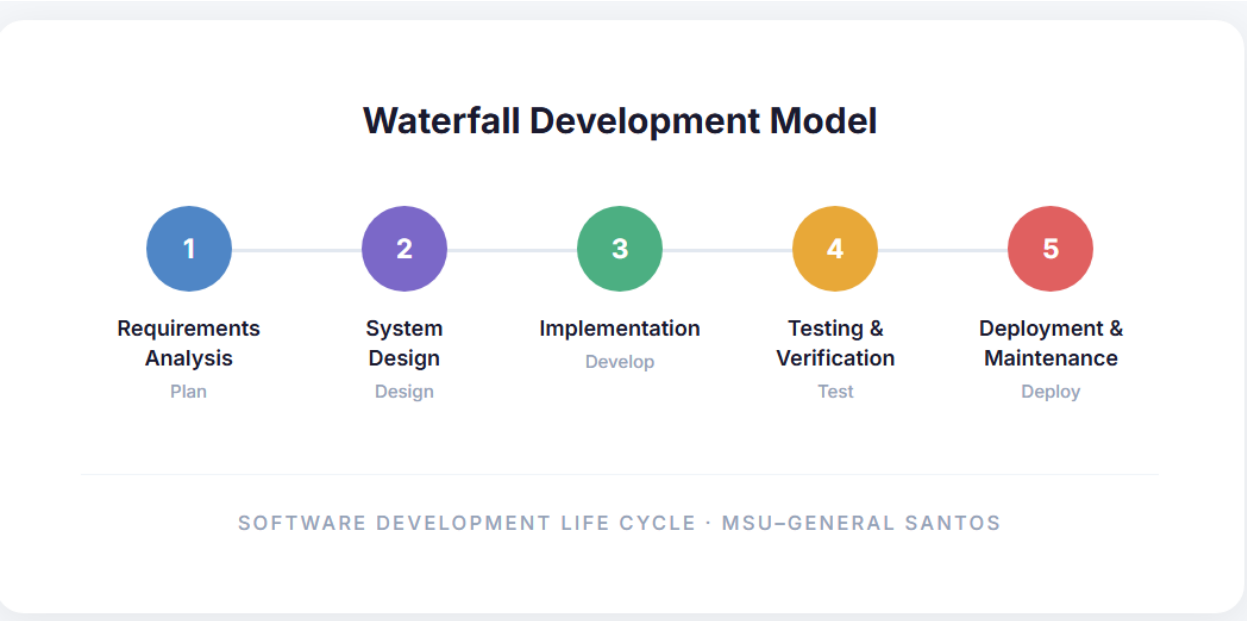


*Figure 3.1 Software Development Cycle*

After testing, it was found that the system acted as desired in all the modules, and the deployment phase commenced. Among the practical benefits of the Waterfall approach in this case is the fact that the deployment is done after all other things have been signed off. It is not possible to risk forcing unfinished features on the users. The actions to be followed in this stage were simple:

1. The XAMPP server setup is configured, which consists of Apache, MySQL, and PHP. phpMyAdmin is installed to administer the database.
2. The system built that has been tested is implemented in the production docket and checked against the initial requirements specification.
3. Students, faculty advisers, and administrators are given end-user training and system documentation.

4. Maintenance Phase aims at carrying out post-deployment monitoring to fix any detected defects, implement updates, and keep the system reliable.

### 3.2.2 DevOps and CI/CD Integration

A GitHub Actions CI/CD pipeline is also incorporated into the development process, and automation of the following steps on each code commit to the main branch:

1. Unit and integration tests are automated and run to ensure the logic of the modules is valid.
2. Artifacts of deployment are Docker images, which are constructed and versioned.
3. The staging environment is adjusted for quality assurance inspection.
4. Each time it is approved, production is updated through zero-downtime rolling deployment.

## 3.3 System Architecture

The AI-based SAIS has a three-level web application framework. This hierarchy design implements separation of concerns and allows each tier to scale independently, as well as create defined responsibility lines between components of the system.

**Hardware Architecture**

The Student Academic Information System (SAIS) is an AI-based client-server architecture that is oriented towards accessibility, security, and plug-in architecture. It is used with regular web browsers on the client computers through laptops or desktop computers, with students, faculty advisers, and administration communicating with their respective dashboards according to their respective roles. The SAIS web application will be hosted on a local application server based on XAMPP installed on a local machine, which runs as an Apache web server, PHP server-side scripting language, and MySQL relational database engine. XAMPP offers a complete server capability that manages the implementation of all the business logic, user authentication, academic risk assessment, and data management processes without the need to have an external hosting infrastructure. Every database administration process, such as schema creation, table administration, query execution, and data import are handled by phpMyAdmin, which is the web-based database management tool that is included with XAMPP. This architecture has the advantage of putting all the presentation, application logic, and data into a single controlled local environment, and thus the system is easy to deploy, maintain, and administer within the institutional environment of the Mindanao State University -General Santos.

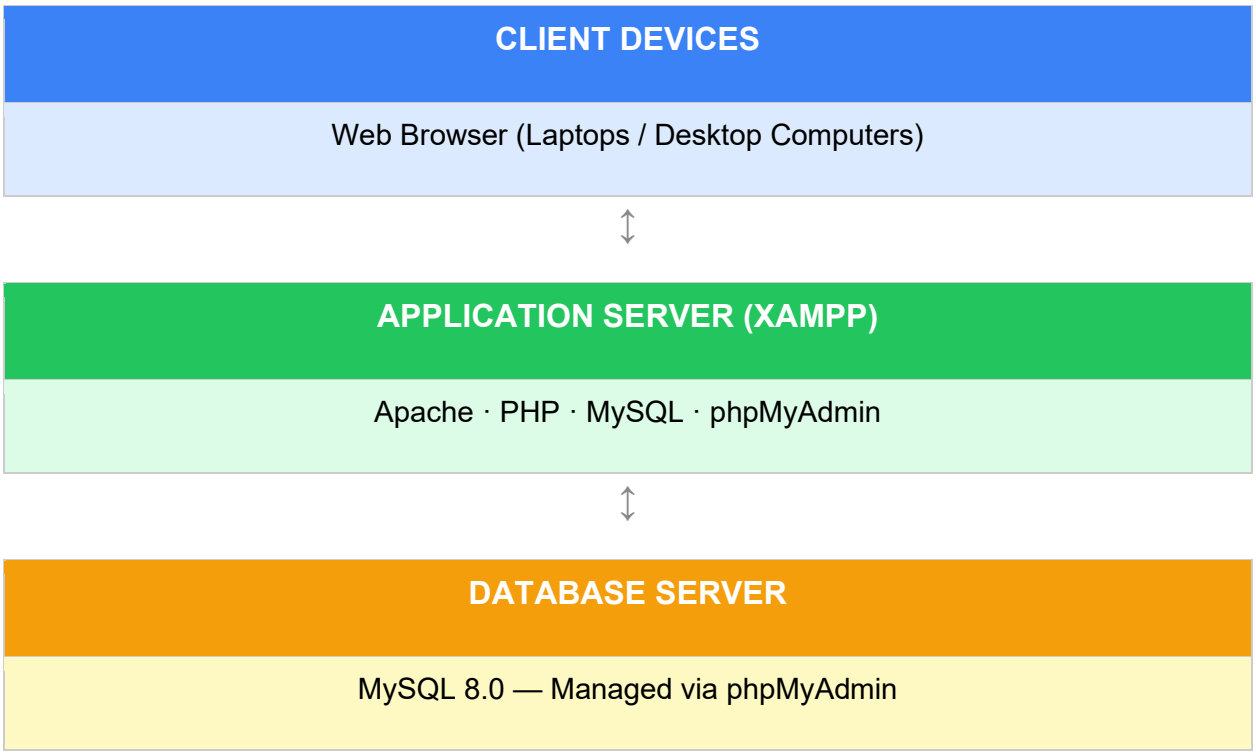


*Figure 4. Hardware Architecture of the AI-Powered SAIS*

**Software Architecture**

The AI-based SAIS has a layered, modular software architecture to guarantee separation of concerns, maintainability, and controlled integration of all system components. The architecture divides the responsibilities for presentation logic, business processing, data management, and output generation into distinct layers, each with a well-defined role. Presentation Layer deals with all communication between the users using a web-based interface, where students can see their grades, attendance, and risk status; faculty advisers can see at-risk students and performance reports; and administrators can see users, courses, and system settings. Application Layer implements the overall business logic of the system that includes the academic classification of risks through a rule-based risk engine, AI chatbot query processing, automated report generation, alert-sending of risks, and audit events. There are all the processing rules and thresholds that are executed at this layer prior to any data being written to or read out of the database. The Data Layer is used to store and retrieve all academic records, user information, risk assessment, chatbot history, and audit trails into a MySQL relational database that is managed using phpMyAdmin. This multi-layered structure guarantees that every functional element of the system can be maintained separately, as well as the integrity of the data that is maintained in all the activities of the system.

**Frontend Architecture**

SAIS frontend architecture is a web-based interface that offers organized user interaction and real-time feedback for all user roles. It includes student, faculty advisor, and administrator roles specific dashboards which are intended to reveal the most pertinent academic information and actions of that role. Students

would get their grades, attendance, their present level of risk, and the AI chatbot. Faculty advisers are shown the list of at-risk students, individual performance reports, and intervention alerts. Administrators deal with users, courses, programs, and settings of the system. User interaction is managed by the frontend using the session-based access control, where a user can view and interact with only the data that they are supposed to have access to, based on their role assignment. The form inputs are created by real-time validation mechanisms that apply data constraints on data before it is submitted to the back-end services. The front-end architecture is aimed at usability, clarity, and guided navigation - decreasing the cognition load on users and providing uniform adherence to back-end validation regulations.

**Backend Architecture**

The backend architecture is a service-based design that divides the core system capabilities into specific processing modules. The modules are to cover a certain functional area:

- Role-based access control and user authentication.
- student grades management (CRUD operations of grades, attendance, and enrollment)
- Risk analytics engine-based on rules - labeling students as either Low, Moderate, or High risk on a per-semester basis.
- AI chatbot processing intent matching with the knowledge base and response generation. • Robotized sending of alerts to at-risk students and the respective faculty advisers. • Data generation of reports and dashboards automatically.
- Logging the audit of the actions of all users and system events. The backend does not permit any mixing of structural computation and data persistence. All the rules and thresholds of risk classification are fully checked, and no results can be recorded in the database. Validation processes are used at various levels, such as post-classification checks, alert threshold checks, and audit log integrity checks to ensure consistent, reliable system behavior.

**Database Architecture**

The SAIS database structure is developed on MySQL, which is managed and administered using phpMyAdmin within the XAMPP setup. The relational data model comprises of twelve fundamental tables that reflect all the academic records, system transactions, and tracking information needed by the system. The following entities are the core entities of the logical data model:

- Users - the stores can store usernames and access status (student, faculty adviser, administrator).
- Programs - academic programs and departments.
- Students - student profiles which are associated with user accounts and academic programs.
- Courses - program-related academic subjects.
- Enrollments student-course enrolment records by semester and by academic year.
- Grades -grade point records correlated with enrollment records.
- Attendance Records - percentage of attendance of students per course per semester.
- Risk Assessments — automated risk classification outcomes by student per semester.

- Knowledge Base - entries of chatbot intention-response.
- Chatbot Logs - student and faculty chatbot logs.
- Alerts Risk and advisory messages sent to students and advisors.
- Audit Logs - record of all the actions of the users and system events (read only).

The structure ensures complete traceability of academic inputs and system outputs - of raw enrollment and grade data to risk classification output results to alert generation and audit log. phpMyAdmin is the graphic interface employed in the entire development of schema creation, table management, query execution, data validation, and importing seed data. The database design is concerned with the structural integrity, metadata traceability and sound data management in the institutional setting of Mindanao State University -General Santos.

### 3.3.1 Architecture Overview

| Architecture Tier | Components | Technology Stack |
|---|---|---|
| **Presentation Layer (Front-End)** | Role-specific dashboards (Student, Adviser, Admin), Chatbot UI, Alert notification center, Grade and attendance viewers | Responsive web framework (SPA), HTML5/CSS3/JavaScript, served via Nginx |
| **Application Layer (Back-End)** | Risk Analytics Engine, Chatbot Processor, Reporting Engine, Auth & Access Control, Alert Dispatcher, Audit Logger | Node.js / Python FastAPI, REST APIs, JWT Authentication, Docker containers |
| **Data Layer** | Transactional records (users, students, enrollments, grades, attendance, alerts, audit logs), Graph relationships (course prerequisites, enrollment sequences) | MySQL 8.0 (relational), ONgDB (graph database), Docker-managed instances |

*Table 3.3. System Architecture – Three-Tier Overview.*

### 3.3.2 Logical Architecture – Component Interactions

In the architectural components, the data and control flow are as follows:

1. The customer browser makes the HTTPS requests to the Nginx reverse proxy.

2. Nginx forwards the requests to the corresponding back-end API service (application container). 3. The API service checks the request by means of JWT, checks the input, and gets the transactional data by means of the MySQL database.
3. In relationship-intensive queries (e.g., prerequisite chains, enrollment graphs), the API will query the ONgDB graph layer.
4. The Risk Analytics Engine will periodically work on the student records stored in MySQL and output risk assessment results to the database.
5. The Alert Module interprets risk outcomes and sends alerts to the concerned students as well as advisers.
6. Auditing: To comply with the regulations, all user activities and system events are tracked to the Audit Log table.

### 3.3.3 Physical Deployment Architecture

| Physical Deployment Configuration | |
|---|---|
| **Web / Proxy Server** | Nginx reverse proxy on Ubuntu 24 LTS – handles SSL termination and load distribution |
| **Application Containers** | Dockerized back-end services (Node.js / FastAPI) with Docker Compose orchestration |
| **Relational Database** | MySQL 8.0 on-premise with automated nightly backups and replication-ready configuration |
| **Graph Database** | ONgDB (Neo4j-compatible) as a Docker container; persistent volume for graph data storage |
| **SSL/TLS** | Let's Encrypt certificate; HTTPS enforced across all routes via Nginx |
| **CI/CD Pipeline** | GitHub Actions: automated test → build → staging → production rollout |
| **Scalability Mechanism** | Nginx upstream load balancing; horizontal container scaling; indexed database queries |

*Table 3.4. Physical Deployment Architecture Configuration.*

### 3.3.4 Security Architecture

Security is embedded at every layer of the system architecture to protect sensitive student academic data:

- Authentication: JSON Web Tokens (JWT) with short expiration windows and secure refresh mechanisms.

- Authorization: Role-based access control (RBAC) restricts data visibility and action permissions per user role.
- Transport Security: TLS 1.3 enforced via Nginx; all API communications encrypted.
- Input Validation: Server-side validation on all API endpoints to prevent SQL injection and XSS attacks.
- Data Privacy: Student risk data accessible only to the student and their assigned adviser; administrator access is audit-logged.
- Compliance: System design aligns with the Data Privacy Act of 2012 (Republic Act No. 10173) of the Philippines.

### 3.4 System Design Specifications

The logic used by the rule-based risk engine is sequential by student by semester:

#### 3.4.1 Module Specifications

| Module | Input | Process | Output |
|---|---|---|---|
| **User Management & Auth** | Login credentials, role assignment | JWT issuance, RBAC validation | Access token, role-based session |
| **Academic Records** | Student data, grades, attendance, enrollment | CRUD validation and persistence | Updated academic records in MySQL |
| **Risk Analytics Engine** | Grades, attendance, failed subject records | Rule-based classification (GPA < 2.0, attendance < 80%, failed ≥ 1) | Risk level (Low / Moderate / High) per student per semester |
| **Alert & Notification** | Risk assessment results | Risk threshold detection, message generation | Push alerts to student and adviser dashboards |
| **AI Chatbot** | Student/faculty text query | Intent classification, knowledge base lookup | Natural language response with academic guidance |
| **Reporting & Dashboards** | Academic records, risk results | Aggregation, chart data generation | Role-specific performance dashboards and PDF reports |

| | | | |
|---|---|---|---|
| **Audit Logging** | User actions, system events | Event capture and timestamp recording | Immutable audit log entries in MySQL |

*Table 3.5. Functional Module Specifications – Input, Process, Output.*

### 3.4.2 Risk Classification Rules

The rule-based risk engine applies the following logic in sequential order per student per semester:

| Step | Evaluation Criterion | Condition | Flag | Risk Tag |
|---|---|---|---|---|
| **1** | GPA Evaluation | GPA < 2.0 | GPA Risk | HIGH |
| **2** | Failed Subject Check | Failed subjects ≥ 1 | Fail Risk | HIGH |
| **3** | Attendance Check | Attendance < 80% in any subject | Attendance Risk | MODERATE |
| **4** | Risk Classification | Two or more flags present | Overall Risk | HIGH |
| **4** | Risk Classification | Exactly one flag present | Overall Risk | MODERATE |
| **4** | Risk Classification | No flags present | Overall Risk | LOW |

*Table 3.6. Rule-Based Academic Risk Classification Logic.*

## 3.5 Data Modeling – Entity-Relationship Design

The architecture of the relational database is MySQL and has twelve basic entities that denote all academic entries, system interactions, and monitoring details. ER model is used to establish the framework of the system in terms of its transactional data layer.

### 3.5.1 Core Entities and Relationships

| Entity (Table) | Primary Key / Foreign Keys | Key Attributes |
|---|---|---|
| **users** | user_id (PK) | username, password_hash, role (student \| faculty_adviser \| admin), created_at |
| **programs** | program_id (PK) | program_name, program_code, description |

| | | |
|---|---|---|
| **students** | student_id (PK), user_id (FK), program_id (FK) | student_number, first_name, last_name, year_level, created_at |
| **courses** | course_id (PK), program_id (FK) | course_code, course_name, units, description |
| **enrollments** | enrollment_id (PK), student_id (FK), course_id (FK) | semester, ay, status (ENROLLED \| COMPLETED \| DROPPED) |
| **grades** | grade_id (PK), enrollment_id (FK) | grade_point, is_failed, date_recorded |
| **attendance_records** | attendance_id (PK), student_id (FK), course_id (FK) | semester, ay, total_sessions, attended_sessions, attendance_percentage |
| **risk_assessments** | risk_id (PK), student_id (FK) | semester, ay, overall_risk, risk_gpa, risk_failed, risk_attendance, explanation, rule_version, assessed_at |
| **knowledge_base** | kb_id (PK) | category, keyword, answer_template, priority |
| **chatbot_logs** | log_id (PK), user_id (FK) | message, response, created_at |
| **audit_logs** | log_id (PK), user_id (FK) | role, action, target_entity, target_id, ip_address, details, created_at |
| **alerts** | alert_id (PK), recipient_id (FK) | role, type, message, is_read, created_at |

*Table 3.7. Entity-Relationship Data Dictionary – Core Tables.*

### 3.5.2 Key ER Relationships

- One user → one student record (1:1 via user_id foreign key).
- One program → many courses and many students (1:N).
- One student → many enrollments across semesters (1:N).
- One enrollment → one grade record (1:1) and one attendance record (1:1).
- One student → many risk assessments (one per semester) (1:N).
- One student → many alerts and many chatbot log entries (1:N).
- One user → many audit log entries (1:N).

### 3.6 Graph Database Modeling (ONgDB)

A framework that has been suggested is to substitute the relational framework. An ONgDB graph database layer is used to complement the relational MySQL model with modeling of complex academic relationships as nodes and edges. This compromised solution offers the performance of transactional databases and relational databases as well as the exploration capability of native graph stores.

#### 3.6.1 Node Definitions

| Node Label | Represents | Key Properties |
|---|---|---|
| **Student** | An individual enrolled student | student_id, student_number, name, year_level, program |
| **Course** | Academic subject or module | course_id, course_code, course_name, units, program |
| **Program** | Academic program/department | program_id, program_code, program_name |
| **RiskAssessment** | Semester risk result per student | risk_id, semester, ay, overall_risk, assessed_at |
| **Adviser** | Faculty adviser | user_id, name, department |

*Table 3.8. ONgDB Graph Node Definitions.*

#### 3.6.2 Edge (Relationship) Definitions

| Relationship | From Node | To Node | Properties / Use |
|---|---|---|---|
| **ENROLLED_IN** | Student | Course | semester, ay, status – models enrollment history |
| **HAS_GRADE** | Student | Course | grade_point, is_failed, date_recorded – links performance to course |
| **ATTENDED** | Student | Course | attendance_percentage, semester – attendance per course |

| | | | |
|---|---|---|---|
| **ASSESSED_AS** | Student | RiskAssessment | Links student to their risk results per semester |
| **PREREQUISITE_OF** | Course | Course | Directed edge modeling prerequisite chain |
| **ADVISED_BY** | Student | Adviser | Assigns adviser to student for intervention tracking |
| **BELONGS_TO** | Course | Program | Links course to its academic program |

*Table 3.9. ONgDB Graph Edge (Relationship) Definitions.*

### 3.6.3 Benefits of the Graph Layer

- Dynamism in traversing course prerequisite chains without costly relational JOINs. Pattern-based identification of hazardous enrollment patterns (e.g., the student who has failed his/her prerequisites several times).
- Mapping of the advisor-student relationship to conduct a specific intervention.
- Graph analytics enabling the analysis of centrality to determine bottleneck courses that have high failure rates.
- An ability to expand to the increasing student populations without architectural modification.

## 3.7 Algorithms and Analytics

The system uses a hybrid of classification logic based on rules and graph traversal algorithms to provide early warning of risks and academic relationship analytics. Any algorithm is aimed at interpretability and efficiency in computing in the academic monitoring setting.

### 3.7.1 Rule-Based Risk Classification Algorithm

The fundamental risk prediction code is a recurring rule assessment engine used on an individual student, per semester. It has a computational complexity of O(n x r) with n being the number of students, and r the number of rules (GPA 3: failed subjects, attendance). It is a deliberate probabilistic algorithm, so that all the risk classifications are auditable and explainable to the student and his/her adviser.

| Risk Classification Algorithm – Pseudo-Code | |
|---|---|
| **Input** | Student record: {GPA, failed_subjects_count, min_attendance_percentage} |
| **Output** | Risk level: LOW \| MODERATE \| HIGH; Explanation string; Rule version |

| | |
|---|---|
| **Step 1** | Evaluate GPA: IF GPA < 2.0 THEN set flag_gpa = HIGH ELSE flag_gpa = LOW |
| **Step 2** | Evaluate failed subjects: IF failed_subjects ≥ 1 THEN flag_fail = HIGH ELSE flag_fail = LOW |
| **Step 3** | Evaluate attendance: IF min_attendance < 80% THEN flag_att = MODERATE ELSE flag_att = LOW |
| **Step 4** | Count flags: flags = COUNT(flag == HIGH or MODERATE) |
| **Step 5** | Classify: IF flags ≥ 2 → HIGH; IF flags == 1 → MODERATE; IF flags == 0 → LOW |
| **Step 6** | Generate explanation string and store result with rule_version and assessed_at timestamp |
| **Complexity** | Time: O(n × r) = O(n) given fixed r = 3 rules; Space: O(n) for result storage |

*Table 3.10. Risk Classification Algorithm – Pseudo-Code and Complexity Analysis.*

### 3.7.2 AI Chatbot Intent Matching Algorithm

The chatbot module compares a weighted algorithm of intent matching with the knowledge base via a keyword. The query of each student or the faculty is tokenized and compared to the knowledge base entry key field. The match score of the entry is used to select the entry with the highest score and return its answer-template as the chatbot answer. When there is no result that meets the confidence level, a fallback response will enable the user to call his or her adviser on his or her own.

- Time Complexity: O(q × k), where q is the number of query tokens and k is the number of knowledge base entries.
- Optimization: Knowledge base entries are pre-indexed and cached in memory, reducing lookup time for high-frequency queries.
- Fallback Mechanism: Queries below the confidence threshold are logged for knowledge base enrichment.

### 3.7.3 Graph Analytics – Traversal Patterns

| Graph Query Type | Use Case | Complexity |
|---|---|---|
| **Shortest Path (Prerequisite Chain)** | Find the minimum course sequence for a student to reach a target course | O(V + E) – Breadth-First Search |
| **Degree Centrality (Course Node)** | Identify courses with the highest enrollment and failure rates (bottleneck courses) | O(V) – count incoming/outgoing edges |
| **Pattern Matching (Risk Path)** | Detect students enrolled in courses they repeatedly fail (high-risk enrollment pattern) | O(V + E) – Cypher pattern match |
| **Subgraph Traversal (Adviser Network)** | Retrieve all students assigned to a specific adviser with their current risk levels | O(d) – degree of adviser node |

*Table 3.11. Graph Analytics – Traversal Patterns and Complexity Analysis.*

# CHAPTER 4 – SUMMARY, FINDINGS, AND CONCLUSION

## 4.1 Summary of Findings from the Analysis

The following summarizes the findings of the analysis. The analysis stage of this research revealed that there have been severe constraints in the current academic information management practices at MSU-General Santos. The manual grade monitoring system meant that the advisers had to peruse through the student records one by one semester after semester, and there was no automatic system to highlight the at-risk students. The performance data were not used with the attendance records, and thus the risk assessment was not conducted holistically. The queries of students and faculty about academic policies were handled manually and used institutional resources and led to delays in delivering information. Conceptual modeling proved that these issues can be solved with the help of a properly designed AI-based system. The IPO framework provided has integrated the student academic records with the risk analytics engine to generate alerts. According to the ER model, all the relationships between the data, that is, between the students, courses, enrollments, grades, attendance, and risk assessments, can be illustrated effectively in a relational schema, and this time this schema should consist of 12 core tables. The ONgDB graph model also expanded the relational design by allowing complex relations traversal, chain analysis of prerequisites as well as adviser-student network mapping that is computationally infeasible in an entirely relational system.

## 4.2 Conclusion According to Models and Frameworks

The suggested AI-based SAIS is highly consistent with the current software engineering standards and the needs of educational technologies. The 3-tier architecture along with Docker containerization and CI/CD automation will offer a powerful, sustainable, and scalable academic monitoring platform. The risk engine that is rule-based ensures transparent interpretable classifications that faculty and advisers can respond to with confidence, and the AI chatbot alleviates the repetitive query load. The hybrid relational-graph data model is a major architectural breakthrough over the traditional student information systems. With the integration of the transactional security of MySQL and the graph navigation functions of ONgDB, the system can handle structured record management alongside complex academic relationship analytics on the same integrated system. The theoretical background of Agile, DevOps, graph theory and, rule-based expert systems together with the conceptual framework of IPO is sufficient to make sure that all decisions made during the design are based on the proven practice of software engineering.

## 4.3 Software Engineering Practice Implications

This paper illustrates that many practical issues can be applied to software engineering within the educational field:

- The advantages of hybrid data architecture (relational and graph-based) are measurably greater than single-paradigm solutions in data-intensive academic settings where relations

are complex. The rule-based expert systems are still a feasible, more desirable choice to machine learning in an institutional risk classification where both interpretability and auditability are prerequisites.

- The integration of CI/CD pipelines in the development of academic systems would severely decrease the risk of implementation and allow responding quickly to changes in institutional needs.
- Introducing AI chatbots directly into student portals is a direct reduction of administrative load alongside a better self-service ability of students that does not need specific support employees.
- When the Agile sprint model is matched with academic calendar milestones, the delivery of the Agile model can be done in phases and validated by the actual student and faculty stakeholders during the active semesters.

# CHAPTER 5 - Future Work and the Innovations

## 5.1 Potential Enhancements

Although the existing system contains a solid base to monitor academics and identify risks at an early stage, the following improvements are recognized to be implemented in a further developmental process: Machine Learning Integration: Rule-based risk prediction by Machine Learning: As historical assessment data of risk accumulates over semesters, risk scoring on probabilistic risk becomes possible with better accuracy.

- Natural Language Understanding (NLU) Upgrade: Repurpose the chatbot as a transformer model (e.g. fine-tuned BERT) to support more complex, contextual student requests.
- Integrated Biometric Attendance: Integrate biometric attendance systems or mobile check-in to remove manual data entry of attendance information and allow real-time updates of risks.
- Predictive Graph Algorithms: Predictive graph-based collaborative filtering to identify students at risk based upon their similarity to high-risk classmates in their enrollment network.
- Automated Knowledge Extraction: Construct an NLP pipeline that will extract and refresh knowledge base entries through official policy documents and advisories of the university.

The architectural design of the AI-powered SAIS supports long-term expansion across multiple dimensions:

## 5.2 Long-Term Expansion Opportunities

- Multi-Campus Deployment: The system should be extended to other campuses of MSU by means of a federated deployment model, where a centralized graph layer should combine inter-campus academic relationship data.
- Interoperability with External Datasets: Integrate with the Commission on Higher Education (CHED) reporting systems and national student performance databases to benchmark institutional performance against regional and national indicators.
- Extended Analytics Dashboards: Build program-level and department-level cohort analytics, which show the trend of academic risk over multiple academic years, which can be used to review the curriculum based on evidence.
- Mobile Application: Build a mobile companion application that will enable students to receive push notifications, access chatbots, view grades on iOS and Android applications.
- Automated Decision-Support Reports: Produce PDF reports ready to go to the advisers with automatically pre-filled intervention recommendations on specific at-risk students based on their risk flags and background.

## REFERENCES


Angles, R., & Gutierrez, C. (2008). Survey of graph database models. *ACM Computing Surveys, 40*(1), 1–39. https://doi.org/10.1145/1322432.1322433

Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A software architect's perspective*. Addison-Wesley.

Buchanan, B. G., & Shortliffe, E. H. (1984). *Rule-based expert systems: The MYCIN experiments of the Stanford Heuristic Programming Project*. Addison-Wesley.

Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. *MIS Quarterly, 36*(4), 1165–1188. https://doi.org/10.2307/41703503

Diestel, R. (2010). *Graph theory* (4th ed.). Springer. https://doi.org/10.1007/978-3-642-14279-6

Pressman, R. S. (2014). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill Education.

Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. Republic of the Philippines. https://privacy.gov.ph/data-privacy-act/

Romero, C., & Ventura, S. (2010). Educational data mining: A review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews, 40*(6), 601–618. https://doi.org/10.1109/TSMCC.2010.2053532

Schwaber, K., & Beedle, M. (2002). *Agile software development with Scrum*. Prentice Hall.

Stonebraker, M., Abadi, D., DeWitt, D. J., Madden, S., Paulson, E., Pavlo, A., & Rasin, A. (2018). MapReduce and parallel DBMSs: Friends or foes? *Communications of the ACM, 53*(1), 64–71. https://doi.org/10.1145/1629175

## APPENDIX A – Sample Code Snippets

### A.1 Rule-Based Risk Classification – Python Pseudo-Code

The following illustrates the core risk classification logic implemented in the Risk Analytics Engine:

```
def classify_risk(student_id, semester, ay, db):
    """Rule-based academic risk classification engine."""
    # Fetch student academic data
    record = db.query(student_id, semester, ay)
    flags = []

    # Rule 1: GPA threshold
    if record.gpa < 2.0:
        flags.append(("GPA", "HIGH"))

    # Rule 2: Failed subjects
    if record.failed_count >= 1:
        flags.append(("FAILED_SUBJECT", "HIGH"))

    # Rule 3: Attendance
    if record.min_attendance_pct < 80.0:
        flags.append(("ATTENDANCE", "MODERATE"))

    # Classify overall risk
    if len(flags) >= 2:
        return RiskResult("HIGH", flags)
    elif len(flags) == 1:
        return RiskResult("MODERATE", flags)
    else:
        return RiskResult("LOW", [])
```

*Appendix A.1. Python Pseudo-Code: Rule-Based Risk Classification Engine.*

### A.2 Sample ONgDB Cypher Queries

The following Cypher queries illustrate key graph analytics operations:

```
// Query 1: Find all HIGH-RISK students and their advisers
MATCH (s:Student)-[:ASSESSED_AS]->(r:RiskAssessment {overall_risk: 'HIGH'})
MATCH (s)-[:ADVISED_BY]->(a:Adviser)
RETURN s.name, s.student_number, r.semester, r.ay, a.name
ORDER BY r.assessed_at DESC;

// Query 2: Prerequisite chain for a course
MATCH path = (start:Course {course_code: 'IT101'})-[:PREREQUISITE_OF*]->(target:Course)
RETURN nodes(path) AS course_sequence, length(path) AS depth
ORDER BY depth;

// Query 3: Identify bottleneck courses (high failure rate)
MATCH (s:Student)-[g:HAS_GRADE]->(c:Course)
WHERE g.is_failed = true
RETURN c.course_code, c.course_name, COUNT(s) AS failure_count
ORDER BY failure_count DESC LIMIT 10;
```

*Appendix A.2. ONgDB Cypher Queries: At-Risk Student Retrieval, Prerequisite Chain, and Bottleneck Course Analysis.*

## APPENDIX B – Data Dictionary

### B.1 MySQL Relational Data Dictionary

Complete attribute definitions for all 12 core tables in the MySQL relational database.

| Table.Column | Data Type | Constraint | Description |
|---|---|---|---|
| **users.user_id** | INT | PK, AUTO_INCREMENT | Unique identifier for each system user |
| users.username | VARCHAR(50) | UNIQUE, NOT NULL | Login username |
| users.password_hash | VARCHAR(255) | NOT NULL | Bcrypt-hashed password |
| users.role | ENUM | NOT NULL | student \| faculty_adviser \| admin |
| **students.student_id** | INT | PK, AUTO_INCREMENT | Unique student identifier |
| students.user_id | INT | FK → users | Links student to their user account |
| students.program_id | INT | FK → programs | Academic program of the student |
| students.student_number | VARCHAR(20) | UNIQUE, NOT NULL | Institutional student ID number |
| students.year_level | INT | NOT NULL | Current year level (1–4) |
| **enrollments.enrollment_id** | INT | PK, AUTO_INCREMENT | Unique enrollment record identifier |
| enrollments.student_id | INT | FK → students | Student who enrolled |
| enrollments.course_id | INT | FK → courses | Course enrolled in |
| enrollments.semester | VARCHAR(10) | NOT NULL | 1st \| 2nd \| Summer |

| | | | |
|---|---|---|---|
| enrollments.ay | VARCHAR(10) | NOT NULL | Academic Year (e.g., 2025-2026) |
| enrollments.status | ENUM | NOT NULL | ENROLLED \| COMPLETED \| DROPPED |
| **grades.grade_id** | INT | PK, AUTO_INCREMENT | Unique grade record identifier |
| grades.enrollment_id | INT | FK → enrollments | Links grade to enrollment record |
| grades.grade_point | DECIMAL(4,2) | NOT NULL | Philippine grade: 1.0 (highest) – 5.0 (failed) |
| grades.is_failed | TINYINT(1) | NOT NULL | 1 if grade_point = 5.0, else 0 |
| grades.date_recorded | DATETIME | NOT NULL | Date and time grade was recorded |
| **risk_assessments.risk_id** | INT | PK, AUTO_INCREMENT | Unique risk assessment identifier |
| risk_assessments.student_id | INT | FK → students | Student assessed |
| risk_assessments.overall_risk | ENUM | NOT NULL | LOW \| MODERATE \| HIGH |
| risk_assessments.rule_version | VARCHAR(10) | NOT NULL | Version of the classification rule set applied |
| risk_assessments.assessed_at | DATETIME | NOT NULL | Timestamp of assessment execution |
| **alerts.alert_id** | INT | PK, AUTO_INCREMENT | Unique alert record identifier |
| alerts.recipient_id | INT | FK → users | User receiving the alert |
| alerts.type | ENUM | NOT NULL | RISK_HIGH \| RISK_MODERATE \| |

| | | | |
|---|---|---|---|
| | | | ATTENDANCE_WARNING \| GENERAL_ADVISORY |
| alerts.is_read | TINYINT(1) | NOT NULL, DEFAULT 0 | 0 = unread; 1 = read by recipient |

*Appendix B.1. MySQL Relational Data Dictionary (Selected Key Attributes).*

B.2 ONgDB Graph Data Dictionary

| Element | Name | Type | Description |
|---|---|---|---|
| **Node** | Student | Graph Node | Represents an enrolled student; properties: student_id, name, year_level, program |
| **Node** | Course | Graph Node | Represents an academic course; properties: course_id, course_code, course_name, units |
| **Node** | Program | Graph Node | Represents an academic program; properties: program_id, program_code, program_name |
| **Node** | RiskAssessment | Graph Node | Represents a risk result; properties: risk_id, semester, ay, overall_risk |
| **Node** | Adviser | Graph Node | Represents a faculty adviser; properties: user_id, name, department |
| **Edge** | ENROLLED_IN | Directed Edge | Student → Course; properties: semester, ay, status |
| **Edge** | HAS_GRADE | Directed Edge | Student → Course; properties: grade_point, is_failed |
| **Edge** | ATTENDED | Directed Edge | Student → Course; properties: attendance_percentage, semester |

| | | | |
|---|---|---|---|
| **Edge** | ASSESSED_AS | Directed Edge | Student → RiskAssessment; links student to risk result |
| **Edge** | PREREQUISITE_OF | Directed Edge | Course → Course; models prerequisite dependency chain |
| **Edge** | ADVISED_BY | Directed Edge | Student → Adviser; assigns adviser for intervention tracking |
| **Edge** | BELONGS_TO | Directed Edge | Course → Program; groups courses by academic program |

*Appendix B.2. ONgDB Graph Data Dictionary – Nodes and Edges.*

## 3% Overall Similarity

Top sources found in the following databases:

- 3% Internet database
- 1% Publications database
- Crossref database
- Crossref Posted Content database

TOP SOURCES

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

| | | |
|---|---|---|
| 1 | digital.lib.usu.edu<br>Internet | <1% |
| 2 | coursehero.com<br>Internet | <1% |
| 3 | classace.io<br>Internet | <1% |
| 4 | repositori.umrah.ac.id<br>Internet | <1% |
| 5 | apppack-app-righttoplay-publics3bucket-veclfraq9ykl.s3.amazonaws...<br>Internet | <1% |
| 6 | repository.machung.ac.id<br>Internet | <1% |
| 7 | penshoppe.com<br>Internet | <1% |
| 8 | files01.core.ac.uk<br>Internet | <1% |
| 9 | ir-library.ku.ac.ke<br>Internet | <1% |

| | | |
|---|---|---|
| 10 | science.gov<br>Internet | <1% |
| 11 | V.P. Iu, L.N. Lamas, Y.-P. Li, K.M. Mok. "Computational Methods in Eng...<br>Publication | <1% |
| 12 | fdocuments.us<br>Internet | <1% |
| 13 | itsfoss.gitlab.io<br>Internet | <1% |
| 14 | mafiadoc.com<br>Internet | <1% |